

1 Linear Systems

1.1 Vector Norms

Vector norm $\|\cdot\|: \mathbb{R}^n \rightarrow \mathbb{R}$.

- Properties:
- PD: $\|x\| \geq 0, \|x\| = 0 \iff x = 0$
 - Homogeneity: $\|\alpha x\| = |\alpha| \cdot \|x\|$
 - Triangle inequality: $\|x + y\| \leq \|x\| + \|y\|$

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p\right)^{1/p},$$
$$\|x\|_\infty = \max |x_i|$$

1.2 Matrix Norms

Matrix norm $\|\cdot\|: \mathbb{R}^{n \times n} \rightarrow \mathbb{R}$.

- Properties:
- PD: $\|A\| \geq 0, \|A\| = 0 \iff A = 0$
 - Homogeneity: $\|\alpha A\| = |\alpha| \cdot \|A\|$
 - Triangle inequality: $\|A + B\| \leq \|A\| + \|B\|$
 - Submultiplicativity: $\|AB\| \leq \|A\| \|B\|$

$$\|A\|_1 = \max_j \sum_{i=1}^n |a_{ij}|,$$
$$\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)},$$
$$\|A\|_F = \sqrt{\sum_{i,j=1}^n |a_{ij}|^2} = \sqrt{\sum_i \sigma_i^2} = \sqrt{\text{tr}(A^T A)},$$
$$\|A\|_\infty = \max_i \sum_{j=1}^n |a_{ij}|$$

1.3 Condition Number / Sensitivity:

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

$\kappa(A)$ measures sensitivity: how much relative perturbations/noise in A or b can change the solution x . If $\kappa(A) \approx 1$, the problem is well-conditioned; if $\kappa(A) \gg 1$, errors are amplified and the matrix is nearly singular/ill-conditioned.

1.4 Gaussian Elimination ($\sim \frac{2}{3}n^3$):

```
for k = 1, ..., n-1 // pivot el.
  for i = k+1, ..., n // rows
    xmult = aik/akk
    aik = 0
  for j = k+1, ..., n // columns
    aij = aij - xmult akj
  end
  bi = bi - xmult bk
end
end
```

Back Substitution ($\sim n^2$):

```
xn = bn/unn
for i = n-1, ..., 1
  s = bi
  for j = i+1, ..., n
    s = s - uijxj
  end
  xi = s/uii
end // solving Ux=b
```

1.5 LU Decomposition

Decomposition of $A \in \mathbb{R}^{n \times n}$ into a lower Δ and an upper Δ matrix:

$$A = LU$$

start with $L = I, U = A$. **Algorithm:**

```
for k = 1, ..., n-1
  for i = k+1, ..., n
    lik = aik/akk
    for j = k+1, ..., n
      aij = aij - likakj
    end
  end
end // same as G.E., but store multipliers in L
```

Solving the system $Ax = b$:

$$A = LU \quad (\sim \frac{2}{3}n^3)$$
$$Ly = b \quad (\sim n^2 - n)$$
$$Ux = y \quad (\sim n^2)$$

1.6 Pivoting

$$PA = LU$$

Partial pivoting: replace the j -th row with the row having the largest $|a_{ij}|, i = j, \dots, n$. Used to avoid zero/small pivots: if a_{jj} is tiny, multipliers $l_{ij} = a_{ij}/a_{jj}$ become large and rounding errors are amplified. **Pivot growth:**

$$\rho(A) = \frac{\max_{i,j} |\hat{u}_{ij}|}{\max_{i,j} |a_{ij}|}.$$

Measures how much entries grow during elimination; small $\rho(A)$ means more stable elimination. For partial pivoting, $\rho(A) \leq 2^{n-1}$. **Permutation matrix:** $P^T = P^{-1}$.

$$P = I_{\text{column } i \leftrightarrow j}, \quad PA = A_{\text{row } i \leftrightarrow j}$$

Row-swap matrix for swapping rows j and k :

$$P_{jk} = I_n - E_{jj} - E_{kk} + E_{jk} + E_{kj}.$$

$P_{jk}A$ swaps rows j and k of A . Swap the 2nd and 3rd rows of matrix A :

$$P_{23}A, \quad P_{23} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Solving the system $Ax = b$:

$$A = P^T LU \quad (\sim \frac{2}{3}n^3)$$
$$Ly = Pb \quad (\sim n^2)$$
$$Ux = y \quad (\sim n^2)$$

1.7 Cholesky ($\sim \frac{1}{3}n^3$). LU decomposition for PD matrices:

$$A = LL^T$$

Algorithm:

```
for k = 1, ..., n // go along diagonal
  lkk = sqrt(akk - sum_{i=1}^{k-1} lki}^2)
  for j = k+1, ..., n // for each el. j below diag.
    for i = 1, ..., k-1
      ajk = ajk - lji}lki
    end
    ljk = ajk/lkk
  end
end
```

Exists when matrix A is:

- Symmetric: $A = A^T$
- PD: $x^T A x > 0 \quad \forall x \neq 0$

Strict row diagonal dominance: $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for every i .

1.8 Iterative Methods For $Ax = b$, direct methods compute the exact solution x^* up to rounding. Iterative methods produce approximations $x^{(k)}$ and stop when the required tolerance is reached:

$$\|\hat{x} - x^*\| \leq \varepsilon \quad \text{or usually} \quad \|Ax^{(k)} - b\| \leq \varepsilon.$$

General form:

$$x^{(k+1)} = R x^{(k)} + c.$$

Useful for large sparse matrices: cheaper memory/work per step, and we can stop as soon as the approximation is accurate enough.

1.9 Spectral Radius

$$\rho(R) = \max_i |\lambda_i(R)|.$$

For $x^{(k+1)} = R x^{(k)} + c$, convergence for every $x^{(0)} \iff \rho(R) < 1$. Smaller $\rho(R)$ means faster convergence: the error is roughly multiplied by $\rho(R)$ each step.

$$e^{(k)} \approx R^k e^{(0)}, \quad \|e^{(k)}\| \sim \rho(R)^k \|e^{(0)}\|.$$

Convergence rate:

$$r(R) = -\log_{10} \rho(R).$$

For error about 10^{-p} , approximate number of steps:

$$m \approx \frac{p}{r(R)}.$$

1.10 Jacobi Iteration ($O(mn)$).

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k-1)} \right), \quad i = 1, \dots, n.$$

$$x_n^{(k)} = \frac{b_n - a_{n,1}x_1^{(k-1)} - \dots - a_{n,n-1}x_{n-1}^{(k-1)}}{a_{nn}}$$

1.11 Gauss-Seidel Component form:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right)$$

More memory-efficient than Jacobi: updates x_i in-place, so it does not need two separate vectors for old/new iterates.

1.12 SOR Relaxed GS iteration with $\omega \in \mathbb{R}$:

$$x_i^{(k)} = (1 - \omega)x_i^{(k-1)} + \omega x_i^{\text{GS}}.$$

$$x_i^{(k)} = (1 - \omega)x_i^{(k-1)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k)} - \sum_{j > i} a_{ij} x_j^{(k-1)} \right).$$

If A is PD and $0 < \omega < 2$, SOR converges.

1.13 Vectors are conjugate

, if $x^T A y = 0$

$$\langle x, y \rangle_A = x^T A y).$$

1.14 Overdetermined Systems

more equations than unknowns:

$$A \in \mathbb{R}^{n \times m}, \quad b \in \mathbb{R}^n, \quad n \gg m$$

1.15 Least Squares Method

$$\hat{x} = \underset{x}{\operatorname{argmin}} \|Ax - b\|$$

The least-squares solution of $Ax = b$ is x , which solves the system $A^T A x = A^T b$. Using Cholesky:

$$B = A^T A, \quad c = A^T b \quad (\sim nm^2)$$

$$B = V V^T \quad (\sim \frac{1}{3}m^3)$$

$$V y = c \quad (\sim m^2)$$

$$V^T \hat{x} = y \quad (\sim m^2)$$

1.16 QR Decomposition

For $A \in \mathbb{R}^{n \times m}, n \geq m$

$$A = QR$$

$$Q^T = Q^{-1}, \quad R \text{ upper } \Delta$$

Solve:

$$Rx = Q^T b$$

For least squares, QR is usually better than normal equations $A^T A x = A^T b$ because forming $A^T A$ squares the condition number:

$$\kappa(A^T A) = \kappa(A)^2.$$

So if A is already poorly conditioned, $A^T A$ is much worse; QR works with A directly.

1.17 Gram-Schmidt

Orthogonalization of the set $\{a_1, \dots, a_n\} \Rightarrow \{q_1, \dots, q_n\}$, such that $L(q_i) = L(a_i)$

$$q_1 = \frac{a_1}{\|a_1\|}, \quad q_k = \frac{a_k - \sum_{j=1}^{k-1} \langle a_k, q_j \rangle q_j}{\|q_k\|}$$

1.18 Householder Reflection

$$H_w = I - 2 \frac{w w^T}{w^T w}, \quad H_w^T = H_w, \quad H_w^2 = I$$

$$w = x + \text{sign}(x_1) \|x\|_2 \cdot e_1$$

$$R = H_k \dots H_1 A, \quad Q = H_1 \dots H_k$$

Geometric meaning: H_w reflects a vector across the hyperplane orthogonal to w . The component parallel to w changes sign, while the perpendicular part is unchanged:

$$x = \alpha w + u, \quad u \perp w \quad \Rightarrow \quad H_w x = -\alpha w + u.$$

Use Householder for QR when numerical stability matters: classical GS can lose orthogonality if columns are nearly linearly dependent. Householder is not necessarily fewer flops than GS, but is more stable and standard for dense QR.

1.19 Singular Value Decomposition (SVD).

For $A \in \mathbb{R}^{n \times m}$, there exist orthogonal $U \in \mathbb{R}^{n \times n}$, $V \in \mathbb{R}^{m \times m}$ and diagonal $\Sigma \in \mathbb{R}^{n \times m}$:

$$U U^T = I_n, \quad V^T V = I_m, \quad \Sigma = \text{diag}(\sigma_1, \dots, \sigma_{\min(n,m)})$$

$$\sigma_1 \geq \dots \geq \sigma_{\min(n,m)} \geq 0, \quad A = U \Sigma V^T.$$

Right $v_i \in \mathbb{R}^m$ are input directions, and left $u_i \in \mathbb{R}^n$ are output directions: $A v_i = \sigma_i u_i$. Singular values are not eigenvalues of A in general:

$$\sigma_i = \sqrt{\lambda_i(A^T A)} \geq 0.$$

Eigenvalues solve $A v = \lambda v$ (same input/output direction, square A); singular values measure stretching $A v_i = \sigma_i u_i$ and also exist for rectangular matrices.

1.20 Moore–Penrose Inverse For $A \in \mathbb{R}^{n \times m}$, the pseudoinverse $A^+ \in \mathbb{R}^{m \times n}$ satisfies:

$$AA^+A = A, \quad A^+AA^+ = A^+, \\ (AA^+)^T = AA^+, \quad (A^+A)^T = A^+A.$$

If $A = U\Sigma V^T$ is SVD, then

$$A^+ = V\Sigma^+U^T, \quad \sigma_i^+ = \begin{cases} 1/\sigma_i, & \sigma_i \neq 0, \\ 0, & \sigma_i = 0. \end{cases}$$

Least squares solution can be written $x = A^+b$.

2 Finding Eigenvalues

2.1 Jordan and Schur Forms Eigenvalue methods try to transform A into a simpler similar matrix, because similar matrices have the same eigenvalues.

Jordan form is the theoretical canonical form:

$$A = SJS^{-1}, \quad J = \text{diag}(J_1, \dots, J_r).$$

Each Jordan block has λ on the diagonal and 1 above it:

$$J_k = \begin{bmatrix} \lambda & 1 & & 0 \\ & \lambda & \ddots & \\ & & \ddots & 1 \\ 0 & & & \lambda \end{bmatrix}.$$

If A has enough independent eigenvectors, then J is diagonal. Otherwise, Jordan blocks appear. Computing Jordan form is numerically unstable, so it is mostly theoretical.

Schur form is the practical replacement for Jordan form:

$$Q^*AQ = T,$$

where Q is unitary and T is upper triangular. The eigenvalues are the diagonal entries of T . For real matrices:

$$U^T AU = T,$$

where U is orthogonal and T is quasi-upper triangular: it has 1×1 blocks for real eigenvalues and 2×2 blocks for complex conjugate pairs. QR iteration moves the matrix toward Schur form using similarity transformations:

$$A_i = Q_i R_i, \quad A_{i+1} = R_i Q_i = Q_i^T A_i Q_i.$$

So eigenvalues stay the same, while A_i becomes more triangular after many iterations.

2.2 Power Method Dominant eigenpair. Input: $A \in \mathbb{R}^{n \times n}$, $x_0 \neq 0$, tol. Output: (x, λ) , where $|\lambda|$ is largest. If x_0 is already an eigenvector and $\|x_0\|_2 = 1$, then

$$Ax_0 = \lambda x_0 \quad \Rightarrow \quad x_0^T Ax_0 = \lambda x_0^T x_0 = \lambda.$$

So estimate eigenvalues by the Rayleigh quotient $\bar{\lambda} = x^T Ax$ for normalized x .

```
x0 = x0 / ||x0||2,    λ0 = x0^T Ax0,    i = 0
while ||Ax_i - λ_i x_i|| > tol
    y_{i+1} = Ax_i
    x_{i+1} = y_{i+1} / ||y_{i+1}||2
    λ_{i+1} = x_{i+1}^T Ax_{i+1}
    i = i + 1
end
```

Each multiplication by A amplifies the dominant eigenvector direction. Convergence rate $\approx |\lambda_2/\lambda_1|^k$.

2.3 Inverse Iteration For a shift $\sigma \approx \lambda$, use the power method on $(A - \sigma I)^{-1}$; obtain the eigenvalue closest to σ . Input: A , $x_0 \neq 0$, shift σ , tol. Output: eigenpair near σ .

```
x0 = x0 / ||x0||2,    λ0 = x0^T Ax0,    i = 0
while ||Ax_i - λ_i x_i|| > tol
    solve (A - σI)y_{i+1} = x_i
    x_{i+1} = y_{i+1} / ||y_{i+1}||2
    λ_{i+1} = x_{i+1}^T Ax_{i+1}
    i = i + 1
end
```

2.4 Orthogonal Iteration Generalization of power method for the first p eigenpairs. Input: A , orthonormal $Z_0 \in \mathbb{R}^{n \times p}$, tol. Output: first p dominant eigenvalues/eigenspace.

```
i = 0,    A0 = Z0^T A Z0,    L0 = lower triangular part of A0
while ||L_i||_F / ||A_i||_F > tol
    Y_{i+1} = A Z_i
    Y_{i+1} = Z_{i+1} R_{i+1}    (QR, Z_{i+1} orthonormal)
    A_{i+1} = Z_{i+1}^T A Z_{i+1}
    L_{i+1} = lower triangular part of A_{i+1}
    i = i + 1
end
```

Columns of Z_i converge to the dominant eigenspace; diagonal of A_i gives the approximated eigenvalues. **2.5 QR Iteration** Computes all eigenvalues. Input: $A \in \mathbb{R}^{n \times n}$, tol. Output: $\lambda_1, \dots, \lambda_n$.

```
i = 0,    A0 = A,    L0 = lower triangular part of A0
while ||L_i||_F / ||A_i||_F > tol
    compute QR factorization A_i = Q_i R_i
    A_{i+1} = R_i Q_i    (same eigenvalues as A_i)
    L_{i+1} = lower triangular part of A_{i+1}
    i = i + 1
end
```

The diagonal of A_i converges to the eigenvalues; with shifts it converges faster.

2.6 Upper Hessenberg Matrix A matrix $H \in \mathbb{R}^{n \times n}$ is **upper Hessenberg** if

$$h_{ij} = 0 \quad \text{for } i > j + 1.$$

So everything below the first subdiagonal is zero:

$$H = \begin{bmatrix} * & * & * & \cdots & * \\ * & * & * & \cdots & * \\ 0 & * & * & \cdots & * \\ 0 & 0 & * & \cdots & * \\ \vdots & \vdots & & \ddots & \\ \vdots & \vdots & & & * \end{bmatrix}.$$

This form is useful because QR iteration preserves it. **2.7 Reduction to Hessenberg Form** Before QR iteration, transform

$$A \longrightarrow H = Q^T A Q.$$

This is an orthogonal similarity transformation, so eigenvalues are preserved. Using Householder reflections, we zero entries below the first subdiagonal, column by column:

$$A_k = Q_k^T A_{k-1} Q_k, \quad k = 1, \dots, n - 2.$$

At step k , take the part of column k below the diagonal:

$$x = A_{k-1}(k+1 : n, k).$$

Build a Householder reflector $\hat{H}_k$ so

$$\hat{H}_k x = \pm \|x\|_2 e_1,$$

therefore entries $k + 2, \dots, n$ in column k become zero. Embed it into the full matrix:

$$Q_k = \begin{bmatrix} I_k & 0 \\ 0 & \hat{H}_k \end{bmatrix}.$$

Multiplying by Q_k^T from the left zeros the column; multiplying by Q_k from the right keeps a similarity transformation, so eigenvalues are unchanged. Cost: $O(n^3)$, but only once.

2.8 Givens Rotations A Givens rotation zeros one component of a 2D vector:

$$v = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad Gv = \begin{bmatrix} \sqrt{x_1^2 + x_2^2} \\ 0 \end{bmatrix}.$$

With

$$c = \frac{x_1}{\sqrt{x_1^2 + x_2^2}}, \quad s = -\frac{x_2}{\sqrt{x_1^2 + x_2^2}}, \\ G = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}.$$

In large matrices, G only mixes two rows/columns. **2.9 QR of an Upper Hessenberg Matrix** For Hessenberg H , QR is cheap because every column has only one subdiagonal entry to eliminate. Use Givens rotations:

$$R = G_{n-1} \cdots G_1 H,$$

where each G_k zeros $h_{k+1,k}$. At step k , build a 2×2 rotation from the vector

$$x = \begin{bmatrix} h_{k,k} \\ h_{k+1,k} \end{bmatrix}, \quad \hat{G}_k x = \begin{bmatrix} * \\ 0 \end{bmatrix}.$$

Embed it into the full matrix so it only mixes rows k and $k + 1$:

$$G_k = \begin{bmatrix} I_{k-1} & 0 & 0 \\ 0 & \hat{G}_k & 0 \\ 0 & 0 & I_{n-k-1} \end{bmatrix}.$$

Apply $G_k H$ to zero the subdiagonal entry. Continue for $k = 1, \dots, n - 1$ to get upper triangular R . Then

$$H = QR, \quad Q^T = G_{n-1} \cdots G_1.$$

2.10 Cost Dense QR iteration costs about $O(n^3)$ per step. For upper Hessenberg matrices:

$$\text{QR factorization cost} = O(n^2),$$

because there are $O(n)$ Givens rotations and each costs $O(n)$. So the total QR process is reduced roughly from

$$O(n^4) \quad \text{to} \quad O(n^3).$$

3 Solving Nonlinear Equations

3.1 Difference of Function Values When $x_a, x_b \rightarrow x^*$, we have

$$\frac{f(x_b) - f(x_a)}{x_b - x_a} \rightarrow f'(x^*)$$

$$|f(x_b) - f(x_a)| \approx |f'(x^*)| |x_b - x_a|$$

3.2 Bisection Finding a zero of function $f(x)$ on interval $[a, b]$, where $f(a) \cdot f(b) < 0$. At each step take the midpoint and keep the half-interval whose endpoints have opposite signs. Stop when the interval is shorter than the prescribed tolerance.

```
for k = 1, 2, ...
    xm = a + (b - a)/2
    if sign(f(xm)) = sign(f(a))
        a = xm
    else
        b = xm
    end
    if |b - a| < tol, stop
end
```

After n steps the interval length is

$$\delta_n = \frac{b - a}{2^n}.$$

For accuracy ε , need $\delta_n \leq \varepsilon$:

$$n \geq \left\lceil \log_2 \frac{b - a}{\varepsilon} \right\rceil.$$

3.3 Tangent Method / Newton's Method Use the tangent at point x_k to find a zero of $f(x)$:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Point x_{k+1} is the intersection of the tangent at x_k with the x-axis. The order of convergence is $r = 2$, if $f'(x^*) \neq 0$.

3.4 Secant Method Approximate the derivative using the two previous points, so we need two initial approximations: x_0, x_1 :

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$$

$$x_{k+1} = x_k - f(x_k) \cdot \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}$$

3.5 Fixed Point Method Rewrite $f(x) = 0$ as an equivalent fixed-point equation

$$g(x) = x.$$

A point x^* with $g(x^*) = x^*$ is a fixed point. Iterate:

$$x_{k+1} = g(x_k)$$

until the tolerance criterion is satisfied. **3.6 Newton's Method** Generalization of the tangent method for nonlinear systems $F(x) = 0$.

$$J(x_k) \Delta x = -F(x_k)$$

Solve the linear system for Δx by any method, then update:

$$x_{k+1} = x_k + \Delta x$$

$$J(x) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}$$

3.7 Broyden’s Method Approximate the Jacobian matrix by B_r :

$$B_{r+1} = B_r + \frac{F(x^{(r+1)})(\Delta x^{(r)})^T}{\|\Delta x^{(r)}\|_2^2}.$$

3.8 Variational Methods Zeros of $f(x)$ are global minima of the function

$$g(x) = \|f(x)\|^2 = \sum_{i=1}^n (f_i(x))^2.$$

Find the minimum iteratively:

$$x^{(r+1)} = x^{(r)} + \lambda_r v_r, \qquad g(x^{(r+1)}) < g(x^{(r)}).$$

3.9 Choosing the Descent Direction

- general: $v_r \not\perp \nabla g(x)$,
- steepest descent: $v_r = -\nabla g(x)$,
- coordinate descent: $v_r = e_1, e_2, \dots, e_n$ in sequence.

3.10 Gauss–Newton Iteration For an overdetermined nonlinear system, use the pseudoinverse of the Jacobian matrix:

$$x^{(r+1)} = x^{(r)} - J_f(x^{(r)})^+ f(x^{(r)}).$$

4 Polynomial Interpolation

Goal: approximate a function $f(x)$ by an easier function $g(x)$.

Find a polynomial $p(x)$ of degree at most $n - 1$ that satisfies:

$$p(x_i) = y_i, \quad \text{for } i = 1, 2, \dots, n$$

This means that the polynomial interpolates the given points (x_i, y_i) .

4.1 Vandermonde System This problem can be written as a linear system:

$$\begin{bmatrix} 1 & x_1 & \cdots & x_1^{n-1} \\ 1 & x_2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \cdots & x_n^{n-1} \end{bmatrix} \cdot \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

4.2 Orthogonal Polynomials The Vandermonde system can be ill-conditioned, so it is better to use an orthogonal basis $g_0, \dots, g_n$:

$$\langle f, g \rangle = \sum_{i=1}^m f(x_i)g(x_i), \qquad \langle g_j, g_k \rangle = 0 \ (j \neq k).$$

$$p(x) = \sum_{i=0}^n a_i g_i(x), \qquad a_i = \frac{\langle f, g_i \rangle}{\langle g_i, g_i \rangle}.$$

4.3 Lagrange Form

$$\ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}, \qquad i = 0, \dots, n.$$

$$p(x) = \sum_{i=0}^n \ell_i(x) y_i.$$

4.4 Newton Form

$$p_n(x) = c_0 + c_1(x - x_0) + \cdots + c_n \prod_{j=0}^{n-1} (x - x_j).$$

The coefficients are divided differences:

$$c_i = f[x_0, \dots, x_i], \qquad i = 0, \dots, n.$$

x	$f[\cdot]$	$f[\cdot, \cdot]$	$f[\cdot, \cdot, \cdot]$
x_0	$f[x_0]$		
x_1	$f[x_1]$	$f[x_0, x_1]$	
x_2	$f[x_2]$	$f[x_1, x_2]$	$f[x_0, x_1, x_2]$
x_3	$f[x_3]$	$f[x_2, x_3]$	$f[x_1, x_2, x_3]$

Newton coefficients are the top diagonal:

$$c_0 = f[x_0], \quad c_1 = f[x_0, x_1], \quad c_2 = f[x_0, x_1, x_2], \dots$$

$$f[x_i] = f(x_i) = y_i, \qquad f[x_i, x_j] = \frac{f(x_j) - f(x_i)}{x_j - x_i}.$$

$$f[x_0, \dots, x_n] = \frac{f[x_1, \dots, x_n] - f[x_0, \dots, x_{n-1}]}{x_n - x_0}.$$

4.5 Gaussian Quadrature Formula Goal: approximate a weighted integral by values of f at smartly chosen nodes:

$$\int_a^b f(x)w(x) \, \mathrm{d}x \approx \sum_{i=1}^n \rho_i f(x_i).$$

Here x_i are nodes and ρ_i are weights. Unlike Newton–Cotes, the nodes are not equidistant; they are chosen so the rule is exact for polynomials of as high degree as possible.

Use orthogonal polynomials for the inner product

$$\langle f, g \rangle_w = \int_a^b f(x)g(x)w(x) \, \mathrm{d}x, \qquad w(x) > 0.$$

If q_n is the n -th orthogonal polynomial, then the Gaussian nodes are its zeros:

$$q_n(x_i) = 0, \qquad i = 1, \dots, n.$$

Weights can be written using Lagrange basis polynomials ℓ_i on these nodes:

$$\rho_i = \int_a^b \ell_i(x)w(x) \, \mathrm{d}x.$$

With n nodes and n weights, Gaussian quadrature is exact for all polynomials up to degree $2n - 1$.

5 Numerical Integration

5.1 Newton–Cotes Rules A special form of least squares, where nodes x_i are chosen equidistantly on interval $[a, b]$:

$$x_i = a + ih, \quad h = \frac{b - a}{mn}, \quad i = 0, \dots, n$$

Trapezoidal Rule ($n = 1, \leq 1$):

$$= \frac{h}{2}(f(a) + f(b)) - \frac{h^3 f''(\xi)}{12}$$

Simpson’s Rule ($n = 2, \leq 3!$):

$$= \frac{h}{3}(f(x_0) + 4f(x_1) + f(x_2)) - \frac{h^5 f''(\xi)}{90}$$

Composite Trapezoidal ($m > 1$):

$$= \frac{h}{2}(f(x_0) + 2f(x_1) + \cdots + 2f(x_{n-1}) +$$

$$f(x_n)) - \frac{(b - a)h^2 f''(\xi)}{12}$$

Composite Simpson ($m > 1$):

$$= \frac{h}{3}(f(x_0) + 4f(x_1) + 2f(x_2) + \cdots +$$

$$2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)) - \frac{(b - a)h^4 f''(\xi)}{280}$$

5.2 Romberg Method $h = (b - a)/n$ is the subinterval width of the composite trapezoidal rule $T(h)$. The error has an expansion in even powers:

$$T(h) = I + c_1 h^2 + c_2 h^4 + \cdots$$

In each new row, double the number of subintervals, so $h_\ell = h_0/2^\ell$. Start with

$$R_{\ell,0} = T(h_\ell)$$

and compute the triangular table:

$$R_{\ell,m} = \frac{4^m R_{\ell,m-1} - R_{\ell-1,m-1}}{4^m - 1}, \qquad 1 \leq m \leq \ell.$$

h	$R_{\ell,0} = T(h_\ell)$	$R_{\ell,1}$	$R_{\ell,2}$
h_0	$R_{0,0}$		
$h_0/2$	$R_{1,0}$	$R_{1,1}$	
$h_0/4$	$R_{2,0}$	$R_{2,1}$	$R_{2,2}$
$h_0/8$	$R_{3,0}$	$R_{3,1}$	$R_{3,2}$

Best approximations are on the top diagonal:

$$R_{0,0}, \quad R_{1,1}, \quad R_{2,2}, \dots$$

Each new column eliminates the next error term: $h^2, h^4, \dots$

6 Differential Equations (DE)

6.1 Definition. DE: equation of the form:

$$F(x, y, y', y'', \dots, y^{(n)}) = 0, \quad y = y(x)$$

The general solution is a family of functions with n parameters. A particular solution is obtained when boundary conditions are given.

Order of a DE: the highest derivative that appears.

6.2 Introducing a New Variable. Convert a higher-order ODE to a first-order system by naming each derivative:

$$u_1 = y, \quad u_2 = y', \quad \dots, \quad u_n = y^{(n-1)}.$$

Then

$$u_1' = u_2, \quad u_2' = u_3, \quad \dots, \quad u_n' = F(x, u_1, \dots, u_n).$$

Example:

$$y'' = F(x, y, y') \Rightarrow \begin{cases} u_1' = u_2, \\ u_2' = F(x, u_1, u_2). \end{cases}$$

For $y'' = y' + 2y$:

$$u_1 = y, \quad u_2 = y', \quad u_1' = u_2, \quad u_2' = u_2 + 2u_1.$$

6.3 Separable First-Order DE Recognize equations where all x terms can go on one side and all t terms on the other:

$$\dot{x} = f(t)g(x), \qquad \dot{x} = \frac{dx}{dt}.$$

Procedure:

$$\frac{dx}{dt} = f(t)g(x) \Rightarrow \frac{dx}{g(x)} = f(t) \, \mathrm{d}t$$

$$\Rightarrow \int \frac{1}{g(x)} \, \mathrm{d}x = \int f(t) \, \mathrm{d}t + C.$$

Then solve the obtained equation for $x(t)$ if possible.

6.4 Linear First-Order DE Standard form:

$$y' + f(x)y = g(x).$$

Use the integrating factor

$$\mu(x) = e^{\int f(x) \, \mathrm{d}x}.$$

Then multiply the whole equation by μ :

$$\mu y' + \mu f y = \mu g \quad \Rightarrow \quad (\mu y)' = \mu g.$$

Integrate and divide by μ :

$$\mu y = \int \mu(x)g(x) \, \mathrm{d}x + C, \qquad y = \frac{1}{\mu} \left(\int \mu g \, \mathrm{d}x + C \right).$$

6.5 Local and Global Error. Local error: the difference between approximation y_i , obtained from approximation y_{i-1} , and $z(x_i)$, the result if the same step were taken from exact $y(x_{i-1})$.

$$l_i = y_i - z(x_i)$$

Global error: the difference between approximation y_i and exact $y(x_i)$ at x_i :

$$g_i = y_i - y(x_i) \leq \sum |l_j|$$

6.6 Order of a Method $p \in \mathbb{N}$, for which:

$$l_i = Ch^{p+1} + O(h^{p+2})$$

The order of a method determines the accuracy of the approximation (higher $p \rightarrow$ better method).

6.7 Euler’s Method Order 1: linear approx. using the tangent at x_i .

Explicit:

$$y_{i+1} = y_i + h_i \cdot f(x_i, y_i)$$

Implicit:

$$y_{i+1} = y_i + h_i \cdot f(x_{i+1}, y_{i+1})$$

6.8 Runge–Kutta 4 (RK4) For the IVP

$y' = f(x, y)$, $y(x_0) = y_0$ and step h :

$$\begin{aligned} k_1 &= f(x_n, y_n) \\ k_2 &= f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \\ k_3 &= f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \\ k_4 &= f(x_n + h, y_n + hk_3) \\ y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$

Idea: estimate the slope at start, two midpoints, and end; then use the weighted average 1 : 2 : 2 : 1.

$$x_{n+1} = x_n + h$$

Order 4: local error $O(h^5)$, global error $O(h^4)$. For systems, use the same formulas with vector y_n and slopes k_i .

7 Appendix

7.1 Line Equation Through two points (x_0, y_0) and (x_1, y_1) :

$$y - y_0 = \frac{y_1 - y_0}{x_1 - x_0}(x - x_0).$$

Slope-intercept form:

$$y = kx + n, \quad k = \frac{y_1 - y_0}{x_1 - x_0}.$$

7.2 Taylor Series Replace a function by a polynomial near point a :

$$\begin{aligned} f(x) &= f(a) + f'(a)(x - a) \\ &+ \frac{f''(a)}{2!}(x - a)^2 + \frac{f'''(a)}{3!}(x - a)^3 + \dots \end{aligned}$$

General form:

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!}(x - a)^k.$$

7.3 Derivatives

- $\frac{1}{x} = -\frac{1}{x^2}$
- $x^n = nx^{n-1}$
- $\sqrt{x} = \frac{1}{2\sqrt{x}}$
- $\sqrt[n]{x} = \frac{1}{n\sqrt[n]{x^{n-1}}}$
- $\sin(ax) = a \cos ax$
- $\cos(ax) = -a \sin(ax)$
- $\tan x = \frac{1}{\cos^2 x}$
- $e^ax = ae^{ax}$
- $a^x = a^x \ln a$
- $x^x = x^x(1 + \ln x)$
- $\ln x = \frac{1}{x}$
- $\log_ax = \frac{1}{x \ln a}$
- $\arcsin x = \frac{1}{\sqrt{1-x^2}}$
- $\arccos x = -\frac{1}{\sqrt{1-x^2}}$
- $\arctan x = \frac{1}{1+x^2}$
- $\operatorname{arccot} x = -\frac{1}{1+x^2}$

7.4 Integrals

- $\int x^a dx = \begin{cases} \frac{x^{a+1}}{a+1} + C & a \neq -1 \\ \ln|x| + C & a = -1 \end{cases}$
- $\int \ln x dx = x \ln x - x + C$
- $\int \frac{1}{\sqrt{x}} dx = 2\sqrt{x} + C$
- $\int e^x dx = e^x + C$
- $\int a^x dx = \frac{a^x}{\ln a} + C$
- $\int \cos(ax) dx = \frac{\sin(ax)}{a} + C$
- $\int \sin(ax) dx = \frac{-\cos(ax)}{a} + C$
- $\int \tan x dx = -\ln|\cos x| + C$
- $\int \frac{dx}{\cos^2 x} = \int \sec^2 x dx = \tan x + C$
- $\int \frac{dx}{\sin^2 x} = \int \csc^2 x dx = -\cot x + C$
- $\int \frac{1}{\sqrt{1-x^2}} dx = \arcsin x + C$
- $\int \frac{dx}{ax+b} = \frac{1}{a} \ln|ax+b| + C$
- $\int \frac{1}{x^2+1} dx = \arctan x + C$
- $\int \frac{dx}{x^2+a^2} = \frac{1}{a} \arctan \frac{x}{a} + C$
- $\int \frac{f'(x)}{f(x)} dx = \ln|f(x)| + C$